

# Table des matières

|                                                                |          |
|----------------------------------------------------------------|----------|
| <b>Guide Flutter — Application Master/Détail avec API REST</b> | <b>1</b> |
| Table des matières                                             | 1        |
| 1. Vue d'ensemble                                              | 1        |
| 2. Documentation officielle à consulter                        | 2        |
| 3. Stack technique recommandée                                 | 3        |
| 4. Configuration du projet                                     | 3        |
| 5. Modèles de données (JSON)                                   | 4        |
| 6. Requêtes HTTP asynchrones avec Dio                          | 5        |
| 7. State Management avec Riverpod                              | 6        |
| 8. Navigation Master/Détail avec go_router                     | 7        |
| 9. Interface utilisateur                                       | 8        |
| 10. Gestion des erreurs et états de chargement                 | 10       |
| 11. Adaptation tablette / mobile                               | 10       |
| 12. Ressources complémentaires                                 | 11       |

## Guide Flutter — Application Master/Détail avec API REST

### Table des matières

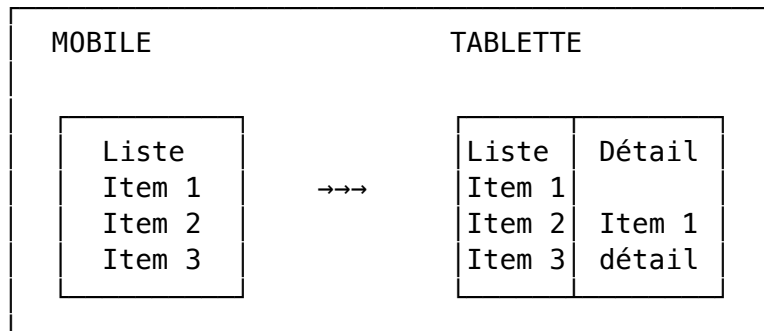
1. Vue d'ensemble
  2. Documentation officielle à consulter
  3. Stack technique recommandée
  4. Configuration du projet
  5. Modèles de données (JSON)
  6. Requêtes HTTP asynchrones avec Dio
  7. State Management avec Riverpod
  8. Navigation Master/Détail avec go\_router
  9. Interface utilisateur
  10. Gestion des erreurs et états de chargement
  11. Adaptation tablette / mobile
  12. Ressources complémentaires
- 

### 1. Vue d'ensemble

Une application **master/détail** est un pattern d'interface classique composé de deux vues :

- **Master** : liste d'éléments (ex. liste d'articles, d'utilisateurs)
- **Détail** : affichage complet d'un élément sélectionné

Sur **mobile**, les deux vues sont affichées séquentiellement (navigation en pile). Sur **tablette**, elles sont affichées côte à côte.



## 2. Documentation officielle à consulter

### Navigation & Layouts adaptatifs

| Sujet                                  | URL                                                                                                                                       |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Layouts adaptatifs (master/détail)     | <a href="https://docs.flutter.dev/ui/adaptive-responsive">https://docs.flutter.dev/ui/adaptive-responsive</a>                             |
| Navigation adaptative (NavigationRail) | <a href="https://docs.flutter.dev/ui/adaptive-responsive/navigation-ui">https://docs.flutter.dev/ui/adaptive-responsive/navigation-ui</a> |
| go_router — routing déclaratif         | <a href="https://pub.dev/packages/go_router">https://pub.dev/packages/go_router</a>                                                       |

### Réseau & JSON

| Sujet                                   | URL                                                                                                                                                                                               |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cookbook : fetch data from the internet | <a href="https://docs.flutter.dev/cookbook/networking/fetch-data">https://docs.flutter.dev/cookbook/networking/fetch-data</a>                                                                     |
| Sérialisation JSON (manuel)             | <a href="https://docs.flutter.dev/data-and-backend/serialization/json">https://docs.flutter.dev/data-and-backend/serialization/json</a>                                                           |
| Sérialisation JSON (code generation)    | <a href="https://docs.flutter.dev/data-and-backend/serialization/json#setting-up-json_serializable">https://docs.flutter.dev/data-and-backend/serialization/json#setting-up-json_serializable</a> |

### State Management

| Sujet                               | URL                                                                                                                                     |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Comparatif des approches            | <a href="https://docs.flutter.dev/data-and-backend/state-mgmt/options">https://docs.flutter.dev/data-and-backend/state-mgmt/options</a> |
| Riverpod (documentation officielle) | <a href="https://riverpod.dev/docs/introduction/getting_started">https://riverpod.dev/docs/introduction/getting_started</a>             |

## Asynchronisme Dart

| Sujet           | URL                                                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------|
| async / await   | <a href="https://dart.dev/libraries/async/async-await">https://dart.dev/libraries/async/async-await</a>                       |
| Future & Stream | <a href="https://dart.dev/libraries/async/futures-error-handling">https://dart.dev/libraries/async/futures-error-handling</a> |

### 3. Stack technique recommandée

| Rôle               | Plugin            | Version | Justification                                |
|--------------------|-------------------|---------|----------------------------------------------|
| Requêtes HTTP      | dio               | 5.x     | Interceptors, retry, cancel tokens, timeouts |
| Sérialisation JSON | json_serializable | 6.x     | Génération automatique de fromJson/toJson    |
| Modèles immuables  | freezed           | 2.x     | Immutabilité, union types, copyWith          |
| State management   | riverpod          | 2.x     | Gestion native des états async (AsyncValue)  |
| Navigation         | go_router         | 14.x    | Deep linking, ShellRoute, standard Google    |

### Plugins de génération de code (dev dependencies)

| Plugin            | Rôle                         |
|-------------------|------------------------------|
| build_runner      | Lance la génération de code  |
| freezed           | Génère les classes immuables |
| json_serializable | Génère les méthodes JSON     |

### 4. Configuration du projet

pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  
  # HTTP  
  dio: ^5.7.0  
  
  # State management  
  flutter_riverpod: ^2.6.1
```

```

riverpod_annotation: ^2.3.5

# Navigation
go_router: ^14.0.0

# Modèles de données
freezed_annotation: ^2.4.4
json_annotation: ^4.9.0

dev_dependencies:
  flutter_test:
    sdk: flutter

# Génération de code
build_runner: ^2.4.13
freezed: ^2.5.7
json_serializable: ^6.8.0
riverpod_generator: ^2.4.3

```

### Lancer la génération de code

```
dart run build_runner build --delete-conflicting-outputs
```

Pour régénérer automatiquement lors du développement :

```
dart run build_runner watch --delete-conflicting-outputs
```

## 5. Modèles de données (JSON)

### Exemple : modèle Article avec freezed + json\_serializable

Fichier lib/models/article.dart :

```

import 'package:freezed_annotation/freezed_annotation.dart';

part 'article.freezed.dart';
part 'article.g.dart';

@freezed
class Article with _$Article {
  const factory Article({
    required int id,
    required String title,
    required String body,
    @JsonKey(name: 'user_id') required int userId,
  }) = _Article;

```

```
factory Article.fromJson(Map<String, dynamic> json) =>
    _$ArticleFromJson(json);
}
```

Après génération (build\_runner), les méthodes fromJson / toJson / copyWith sont disponibles automatiquement.

### Exemple de JSON attendu depuis l'API

```
[
  {
    "id": 1,
    "title": "Premier article",
    "body": "Contenu de l'article...",
    "user_id": 42
  }
]
```

## 6. Requêtes HTTP asynchrones avec Dio

### Configuration du client Dio

Fichier lib/services/api\_client.dart :

```
import 'package:dio/dio.dart';

class ApiClient {
  static const String _baseUrl = 'https://jsonplaceholder.typicode.com';

  final Dio _dio = Dio(
    BaseOptions(
      baseUrl: _baseUrl,
      connectTimeout: const Duration(seconds: 10),
      receiveTimeout: const Duration(seconds: 10),
      headers: {'Content-Type': 'application/json'},
    ),
  );

  Future<List<Map<String, dynamic>>> getArticles() async {
    final response = await _dio.get('/posts');
    return List<Map<String, dynamic>>.from(response.data);
  }

  Future<Map<String, dynamic>> getArticleById(int id) async {
    final response = await _dio.get('/posts/$id');
    return response.data as Map<String, dynamic>;
  }
}
```

```
}  
}
```

## Fonctionnement asynchrone

Le mot-clé `async` / `await` permet d'attendre le résultat d'une opération longue (requête réseau) sans bloquer l'interface :

```
// Sans await → l'exécution continue immédiatement (Future non résolu)  
final future = apiClient.getArticles();  
  
// Avec await → on attend le résultat avant de continuer  
final articles = await apiClient.getArticles();
```

---

## 7. State Management avec Riverpod

Riverpod gère les trois états d'une requête asynchrone via `AsyncValue` : - `AsyncLoading` — requête en cours - `AsyncData` — données reçues - `AsyncError` — erreur survenue

### Provider de la liste (master)

Fichier `lib/providers/articles_provider.dart` :

```
import 'package:riverpod_annotation/riverpod_annotation.dart';  
import '../models/article.dart';  
import '../services/api_client.dart';  
  
part 'articles_provider.g.dart';  
  
@riverpod  
Future<List<Article>> articles(ArticlesRef ref) async {  
  final client = ApiClient();  
  final data = await client.getArticles();  
  return data.map((json) => Article.fromJson(json)).toList();  
}
```

### Provider du détail

```
@riverpod  
Future<Article> articleDetail(ArticleDetailRef ref, int id) async {  
  final client = ApiClient();  
  final data = await client.getArticleById(id);  
  return Article.fromJson(data);  
}
```

## 8. Navigation Master/Détail avec go\_router

### Configuration du routeur

Fichier lib/router.dart :

```
import 'package:go_router/go_router.dart';
import 'screens/articles_list_screen.dart';
import 'screens/article_detail_screen.dart';

final router = GoRouter(
  initialLocation: '/articles',
  routes: [
    GoRoute(
      path: '/articles',
      builder: (context, state) => const ArticlesListScreen(),
      routes: [
        GoRoute(
          path: ':id',
          builder: (context, state) {
            final id = int.parse(state.pathParameters['id']!);
            return ArticleDetailScreen(id: id);
          },
        ),
      ],
    ),
  ],
);
```

### Utilisation dans main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'router.dart';

void main() {
  runApp(const ProviderScope(child: MyApp()));
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      routerConfig: router,
      title: 'Master/Détail Flutter',
    );
  }
}
```

```
}  
}
```

## Navigation vers le détail

```
// Depuis la liste, naviguer vers l'article id=3  
context.go('/articles/3');  
  
// Ou avec push (empile sur la navigation actuelle)  
context.push('/articles/3');
```

---

## 9. Interface utilisateur

### Écran Master (liste)

Fichier lib/screens/articles\_list\_screen.dart :

```
import 'package:flutter/material.dart';  
import 'package:flutter_riverpod/flutter_riverpod.dart';  
import 'package:go_router/go_router.dart';  
import '../providers/articles_provider.dart';  
  
class ArticlesListScreen extends ConsumerWidget {  
  const ArticlesListScreen({super.key});  
  
  @override  
  Widget build(BuildContext context, WidgetRef ref) {  
    final asyncArticles = ref.watch(articlesProvider);  
  
    return Scaffold(  
      appBar: AppBar(title: const Text('Articles')),  
      body: asyncArticles.when(  
        loading: () => const Center(child: CircularProgressIndicator()),  
        error: (error, _) => Center(child: Text('Erreur : $error')),  
        data: (articles) => ListView.builder(  
          itemCount: articles.length,  
          itemBuilder: (context, index) {  
            final article = articles[index];  
            return ListTile(  
              title: Text(article.title),  
              subtitle: Text('Article #${article.id}'),  
              trailing: const Icon(Icons.chevron_right),  
              onTap: () => context.go('/articles/${article.id}'),  
            );  
          },  
        ),  
      ),  
    );  
  }  
}
```



```

    ),
  );
}
}

```

## Écran Détail

Fichier lib/screens/article\_detail\_screen.dart :

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../providers/articles_provider.dart';

class ArticleDetailScreen extends ConsumerWidget {
  final int id;

  const ArticleDetailScreen({super.key, required this.id});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final asyncArticle = ref.watch(articleDetailProvider(id));

    return Scaffold(
      appBar: AppBar(title: const Text('Détail')),
      body: asyncArticle.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (error, _) => Center(child: Text('Erreur : $error')),
        data: (article) => Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                article.title,
                style: Theme.of(context).textTheme.headlineSmall,
              ),
              const SizedBox(height: 16),
              Text(article.body),
            ],
          ),
        ),
      ),
    );
  }
}

```

## 10. Gestion des erreurs et états de chargement

Riverpod gère les états de manière déclarative avec `.when()` :

```
asyncValue.when(  
  loading: () => const CircularProgressIndicator(),  
  error: (error, stackTrace) => ErrorWidget(error),  
  data: (data) => MyWidget(data: data),  
);
```

### Relancer une requête en cas d'erreur

```
// Dans le widget d'erreur  
ElevatedButton(  
  onPressed: () => ref.invalidate(articlesProvider),  
  child: const Text('Réessayer'),  
)
```

### Indicateur de rechargement (pull-to-refresh)

```
RefreshIndicator(  
  onRefresh: () async => ref.invalidate(articlesProvider),  
  child: ListView.builder(...),  
)
```

---

## 11. Adaptation tablette / mobile

Pour afficher les deux panneaux côte à côte sur tablette :

Fichier `lib/screens/adaptive_layout_screen.dart` :

```
import 'package:flutter/material.dart';  
  
class AdaptiveLayoutScreen extends StatefulWidget {  
  const AdaptiveLayoutScreen({super.key});  
  
  @override  
  State<AdaptiveLayoutScreen> createState() => _AdaptiveLayoutScreenState();  
}  
  
class _AdaptiveLayoutScreenState extends State<AdaptiveLayoutScreen> {  
  int? _selectedId;  
  
  static const double _tabletBreakpoint = 600.0;  
  
  bool get _isTablet =>  
    MediaQuery.of(context).size.width >= _tabletBreakpoint;
```

```

@override
Widget build(BuildContext context) {
  if (_isTablet) {
    return Row(
      children: [
        SizedBox(
          width: 300,
          child: ArticlesListPanel(
            onSelected: (id) => setState(() => _selectedId = id),
            selectedId: _selectedId,
          ),
        ),
        const VerticalDivider(width: 1),
        Expanded(
          child: _selectedId != null
            ? ArticleDetailPanel(id: _selectedId!)
            : const Center(child: Text('Sélectionnez un article')),
        ),
      ],
    );
  }

  // Mobile : navigation classique via go_router
  return const ArticlesListScreen();
}

```

## 12. Ressources complémentaires

### Exemples et tutoriels

- [Flutter cookbook](#) — recettes officielles
- [Riverpod examples](#) — exemples officiels
- [go\\_router examples](#)

### Outils de développement

| Outil                           | Usage                                       |
|---------------------------------|---------------------------------------------|
| <a href="#">pub.dev</a>         | Recherche de packages Flutter/Dart          |
| <a href="#">JSONPlaceholder</a> | API REST factice pour les tests             |
| <a href="#">Hoppscotch</a>      | Tester les endpoints d'une API              |
| Flutter DevTools                | Profiling, inspection de l'arbre de widgets |

## Structure de projet recommandée

```
lib/
├── main.dart
├── router.dart
├── models/
│   ├── article.dart
│   ├── article.freezed.dart    ← généré
│   └── article.g.dart         ← généré
├── services/
│   └── api_client.dart
├── providers/
│   ├── articles_provider.dart
│   └── articles_provider.g.dart ← généré
└── screens/
    ├── articles_list_screen.dart
    ├── article_detail_screen.dart
    └── adaptive_layout_screen.dart
```

---

*Document rédigé pour le BUT Informatique — SAÉ S4 — 2025/2026*